Chapter 1

Reinforcement Learning

This chapter introduces the fundamental concepts of reinforcement learning, providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the standard references in reinforcement learning, in particular the works of Sutton and Barto [Sutton and Barto, 2018]. Its structure is as follows:

- Historical developments: an overview of the key ideas and milestones that led to the emergence of reinforcement learning as a unified field.
- Markov Decision Processes (MDPs), policies, and value functions: a formalization of reinforcement learning problems through MDPs, together with the respective roles of the policy (actor) and value estimation (critic).
- Actor and Critic learning: an explanation of how the actor and critic are jointly trained through interaction with the environment.
- Policy evaluation: a discussion of how learned policies are assessed and compared.

1.1 Historical Developments

Reinforcement learning is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

1.1.1 Behavioral foundations (1900–1950)

The origins of reinforcement learning can be traced back to 20th-century animal psychology studies. These studies aimed to understand how behaviors are formed and modified in animals. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be reinforced and repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency [Thorndike, 1911].

This perspective was later developed within behaviorism. In this school of psychology, reinforcement and punishment are used to gradually shape behavior through repeated interaction with the environment [Skinner, 2019]. The term "reinforcement" in the context of learning comes from this school of thought. This perspective in behaviorism influenced early thinking in computing and artificial intelligence. It led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a "pleasure–pain system" [Turing, 2004].

Although these studies did not involve formal mathematical models, they introduced key ideas that later became central to reinforcement learning, including trial-and-error learning and adaptation driven by environmental feedback.

1.1.2 Sequential decision-making (1950–1970)

A second key foundation of reinforcement learning comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, an agent is the decision-making entity that interacts with the environment by selecting actions. A policy is the function that maps an observed state to the action chosen by the agent. In this field, the objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDPs), which provide a mathematical model for sequential decision-making under uncertainty [Howard, 1960]. MDPs are defined by four main components: states, actions, rewards, and transition dynamics. A state represents the current situation of the environment at a given time. An action is a choice available to the policy in a given state. A reward is a scalar signal that provides feedback after each action. Transition dynamics describe how the environment evolves from one state to another.

One of the earliest approaches in this framework is dynamic programming [Bellman, 1957]. This method relies on a recursive decomposition of sequential decision problems via the principle of optimality. By solving each subproblem corresponding to a state, dynamic programming computes the optimal value function and, from it, derives the optimal policy. However, classical dynamic programming methods rely on the assumption that the environment dynamics are fully known. This means that the full state space and all transition relationships must be explicitly modeled, which quickly becomes computationally intractable in realistic problems.

1.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference (TD) learning methods were introduced [Sutton, 1988]. TD learning combines ideas from Monte Carlo methods and dynamic programming by updating predictions based on other learned estimates, rather than waiting for final outcomes.

This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations and forms a key building block of reinforcement learning algorithms.

Building on these new ideas, Q-learning was introduced, showing that optimal action-value functions can be learned directly from interaction with the environment without requiring a model of its dynamics [Watkins, 1989, Watkins and Dayan, 1992]. The publication of the foundational reference work [Sutton and Barto, 2018] contributed to the formalization of reinforcement learning as a coherent field.

1.1.4 Consolidation of reinforcement learning (2000–2013)

A key limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDPs, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced, allowing these quantities to be represented using parametric models such as linear approximators or perceptron-like architectures [Barto et al., 1983, Tsitsiklis and Van Roy, 1996]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space.

The introduction of Deep Q-Networks (DQN), which demonstrated that deep Neural Network (NN) could be used to approximate value functions at scale while maintaining stability [Mnih et al., 2013]. By combining Q-learning with deep NN architectures, DQN successfully extended value-based reinforcement learning to high-dimensional input spaces such as raw pixels in Atari games.

1.1.5 Deep reinforcement learning (2013–2026)

The introduction of deep NN into reinforcement learning marked a major turning point in the field. Building on this idea, a series of algorithms significantly improved scalability. Deep Deterministic Policy Gradient (DDPG) extended actor–critic methods to continuous action spaces [Lillicrap et al., 2019]. Proximal Policy Optimization (PPO) later became a standard algorithm in reinforcement learning due to its simplicity, stability, and ability to perform in both continuous and discrete action spaces [Schulman et al., 2017].

These advances established a practical foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NN, reinforcement learning, and tree search could reach superhuman performance in the game of Go [Silver et al., 2017]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [Samvelyan et al., 2019]. Similarly, in Dota 2, large-scale multi-agent reinforcement learning systems such as OpenAI Five showed that self-play combined with deep policy optimization can achieve professional-level performance in highly dynamic and stochastic environments [OpenAI et al., 2019].

More recently, reinforcement learning has expanded beyond its traditional application domains, such as robotics, video games, and combinatorial optimiza-

tion. With the emergence of increasingly agentic views of large language models, it has become possible to train these models to perform specific tasks using reinforcement learning-based techniques. In particular, reinforcement learning from human feedback (RLHF) has become a key method for aligning large language models with human preferences by combining reward modeling and policy optimization [Ouyang et al., 2022]. Similarly, DeepSeekMath shows how reinforcement learning can enhance the mathematical reasoning abilities of large language models [Shao et al., 2024]. It demonstrates that an artificial agent can learn to reason in natural language in order to accomplish a given task. The skills acquired during this learning process also transfer to other similar tasks, leading to improved overall performance. For instance, training a large language model with reinforcement learning on mathematical problem-solving can also enhance its programming capabilities.

Originally inspired by biological learning processes, reinforcement learning is now closer than ever to this initial intuition. Agents learn through interaction, adapt via trial and error, and progressively improve their behavior based on experience rather than explicit supervision. Combined with deep neural architectures, this paradigm strengthens the analogy with natural learning mechanisms, further aligning computational learning processes with those observed in living organisms. Thanks to these advances, reinforcement learning systems can learn internal representations of their environment and generalize beyond previously encountered situations. These advances contribute to the ongoing progress toward artificial general intelligence, and more than ever, reinforcement learning stands as one of the core paradigms for such systems [Silver et al., 2021].

Bibliography

[Barto et al., 1983] Barto, A. G., Sutton, R. S., and Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics, 13(5):835–846.

[Bellman, 1957] Bellman, R. E. (1957). Dynamic programming.

[Howard, 1960] Howard, R. A. (1960). Dynamic programming and markov processes. MIT Press.

[Lillicrap et al., 2019] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2019). Continuous control with deep reinforcement learning.

[Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

[OpenAI et al., 2019] OpenAI, :, Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., d. O. Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning.

[Ouyang et al., 2022] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback.

[Samvelyan et al., 2019] Samvelyan, M., Rashid, T., de Witt, C. S., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J., and Whiteson, S. (2019). The starcraft multi-agent challenge.

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.

[Shao et al., 2024] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models.

Compiled on 23/6/2026 at 10:59

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm.

[Silver et al., 2021] Silver, D., Singh, S., Precup, D., and Sutton, R. S. (2021). Reward is enough. Artificial intelligence, 299:103535.

[Skinner, 2019] Skinner, B. F. (2019). The behavior of organisms: An experimental analysis. BF Skinner Foundation.

[Sutton, 1988] Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine Learning, 3:9–44.

[Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press, 2nd edition.

[Thorndike, 1911] Thorndike, E. L. (1911). Animal intelligence: Experimental studies. The Psychological Review Monograph Supplement, 8(4).

[Tsitsiklis and Van Roy, 1996] Tsitsiklis, J. and Van Roy, B. (1996). Analysis of temporal-diffference learning with function approximation. Advances in neural information processing systems, 9.

[Turing, 2004] Turing, A. (2004). Intelligent machinery (1948). B. Jack Copeland, page 395.

[Watkins, 1989] Watkins, C. J. C. H. (1989). Learning from delayed rewards. PhD Thesis, University of Cambridge.

[Watkins and Dayan, 1992] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. Machine Learning, 8:279–292.